ORGNZ-08: Grail Segments

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 8 of 10 | **Created:** January 2026 | **Last Updated:** 10/12/2026

Overview

Grail segments are logical groupings of data that enable real-time filtering across massive datasets without creating thousands of individual rules. Segments bring business context to observability data and are consistently available across the Dynatrace platform.

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`storage:filter-segments:read`, `storage:logs:read`, `storage:entities:read`
Knowledge	Completed ORGNZ-07 (Advanced Permissions)
Data	At least 1 hour of log, span, and entity data

Table of Contents

1. Segments vs Buckets
2. Segment Use Cases
3. Segment Structure
4. Segment Limits

5. Segment Design Patterns
6. Leveraging OneAgent Enrichments
7. Entity Relationship Strategies
8. Testing Segments
9. Segments and Access Control
10. Segment Design Checklist
11. Bucket-Based Segment Filtering

Learning Objectives

By the end of this notebook, you will:
- Understand segments vs buckets
- Design effective segment rules
- Use dynamic variables in segments
- Apply entity relationships for comprehensive filtering

Segments vs Buckets

Aspect	Buckets	Segments
Purpose	Physical data storage	Logical data filtering
Retention	Controls data retention	No impact on retention
Data types	One type per bucket	Multiple types per segment
Access control	Can be used with IAM	Data filtering only
Cost impact	Direct storage costs	No direct cost impact
Query scope	Storage-level partitioning	Query-time filtering

> **Key insight**: Buckets store data; segments filter it. Use buckets for retention and cost allocation, segments for dynamic data views.
>
> **Performance benefit**: Applying a segment also reduces the **scanned bytes** of your queries. Since segments inject filter conditions at query time, only matching data is

> scanned — meaning fewer bytes read, lower cost, and faster results. This is especially impactful on high-volume buckets where unfiltered queries might hit the default 500 GB scan limit (`scanLimitGBytes`).

Segment Use Cases

1. Team-Based Views

```
Segment: "Platform Team"
Includes: Infrastructure hosts, Kubernetes clusters, platform services
Benefit: Team sees only relevant data, faster troubleshooting
```

2. Application Isolation

```
Segment: "Checkout Application"
Includes: Checkout service, related databases, frontend components
Benefit: Complete application view across logs, traces, metrics
```

3. Environment Filtering

```
Segment: "Production Only"
Includes: Entities with environment=production tag
Benefit: Focused production monitoring without dev/test noise
```

Segment Structure

```
Segment:
  name: "segment-identifier"
  displayName: "Human Readable Name"
  description: "What this segment filters"

  includes:
    - type: logs
      filter: "host.name starts-with 'prod-'"

    - type: dt.entity.host
      filter: "tags contains 'environment:production'"

    - type: spans
      filter: "service.name == 'checkout'"

  variables:
    - name: environment
      values: ["production", "staging"]
```

Key behavioral rules:

- **Include blocks are ORed together** — data matching *any* include is in scope; display order in the UI does not affect which data is included
- **One include per data/entity type** — you cannot add two log includes; use AND/OR within a single filter condition instead
- **Evaluated at query time** — filter conditions inject into every query that uses the segment; complex conditions increase query overhead. Preprocess fields via **OpenPipeline** to simplify conditions and improve performance

Segment Limits

Limit	Value	Notes
Segments per environment	10,000	Tenant-wide ceiling
Maximum includes (rules) per segment	20	Plan rule complexity accordingly
Include blocks per data/entity type	1	Cannot have multiple rules for same type

Limit	Value	Notes
Expressions per filter condition	1 min / 10 max	Maximum conditions per filter
Values per variable	10,000	Maximum values in the variable dropdown
Selected variable values per segment	100	Classic entities
Segments applied per query	10	Cannot stack unlimited segments
Additional include per classic entity type + relationship	1	One relationship hop only

Operators and Wildcards

Segment filter conditions offer only two operators: `=` **and** `in`. There is no separate `contains` operator — substring and suffix matching is expressed by placing a **wildcard** `*` **inside the value**. Conditions are written in the **filter-field grammar** of the segment editor (`key = value` , `key in (value1, value2)`), not in DQL: the DQL function form `in(field, {"a", "b"})` is a different language.

Matching	How to express it	Where it works
Equals	`field = "value"`	Everywhere
Set membership	`field in ("a", "b")`	Everywhere — exact values only; `*` inside the list is not a wildcard
Starts-with	`field = "prefix*"`	Signal data; classic `entity.name`
Contains	`field = "*text*"`	Signal data — not classic entity includes
Ends-with	`field = "*suffix"`	Signal data — not classic entity includes

> **Important**: on **classic entity** includes, `entity.name` supports **starts-with only** (`entity.name = "payment*"`); contains and ends-with are not supported on any classic-entity property, and every other property is exact `equals` only. Signal-data includes (logs, spans, events, metrics) have no such restriction. Tag conditions are a

> separate case — a tag filter tests **membership in the tag set**, not a substring of a string field — so verify tag-based entity includes against your own tenant before relying on wildcard behavior there.
>
> An asterisk may also follow a variable name in a starts-with condition — for example `foo = $bar*` — which is what makes variable-driven segments practical.
>
> **Sources:** Segment limits (DT docs) — *"Only operators `=`, and `in()` are supported."*; classic entities: *"starts-with, contains, and ends-with are not supported"*, *"entity.name supports starts-with"*, *"Other properties only support equals"*. Filter field (DT docs) — `in` *"matches one or more values in a list of values"*; *"Wildcards are not supported within the in () operator."*

Segment Design Patterns

Pattern 1: Team-Based Segment

```
name: team-platform
displayName: "Platform Team"
description: "All infrastructure and platform services"

includes:
  - type: dt.entity.host
    filter: "tags contains 'team:platform'"

  - type: logs
    filter: "dt.host_group.id starts-with 'platform-'"

  - type: spans
    filter: "service.name starts-with 'platform-'"
```

Pattern 2: Application-Based Segment

```
name: app-checkout
displayName: "Checkout Application"
description: "Checkout service and dependencies"

includes:
  - type: dt.entity.service
    filter: "tags contains 'app:checkout'"

  - type: logs
    filter: "k8s.namespace.name == 'checkout'"

  - type: spans
    filter: "service.name starts-with 'checkout-'"
```

Pattern 3: Environment-Based Segment

```
name: env-production
displayName: "Production Environment"
description: "Production workloads only"

variables:
  - name: env
    values: ["production", "prod"]

includes:
  - type: dt.entity.host
    filter: "tags contains 'environment:${env}'"

  - type: logs
    filter: "dt.host_group.id starts-with 'prod-'"
```

Leveraging OneAgent Enrichments

OneAgent automatically enriches signals with metadata for precise filtering:

Enrichment	Source	Example Use
`dt.host_group.id`	Host group configuration	Filter by host group

Enrichment	Source	Example Use
`k8s.namespace.name`	Kubernetes namespace	Filter by namespace
`k8s.cluster.name`	Kubernetes cluster	Filter by cluster
`cloud.provider`	Cloud detection	Filter by AWS/Azure/GCP
`cloud.region`	Cloud region	Filter by region
`tags`	Entity tags	Filter by custom tags

Host Group Filtering Example

```
includes:
  - type: logs
    filter: "dt.host_group.id == 'prod-web-tier'"

  - type: dt.entity.host
    filter: "hostGroup == 'prod-web-tier'"
```

Entity Relationship Strategies

> **Backward compatibility note**: Entity-to-entity relationship traversal in segments is supported only for **classic entities** (application, service, process, host, data center). Smartscape node-based segments should use direct field filters on Primary Grail Fields instead.

Traversal rules:
- Only **one relationship hop** is supported per include block
- Multiple **parallel** relationships from the same originating entity type are allowed

Top-Down (Host-Centric)

Start from hosts and include related entities:

```
includes:
  - type: dt.entity.host
    filter: "tags contains 'team:platform'"

  - type: dt.entity.process_group
    relationship: "runsOn dt.entity.host"

  - type: dt.entity.service
    relationship: "runsOnProcessGroup dt.entity.process_group"
```

Bottom-Up (Service-Centric)

Start from services and include supporting infrastructure:

```
includes:
  - type: dt.entity.service
    filter: "tags contains 'app:checkout'"

  - type: dt.entity.process_group
    relationship: "hostsService dt.entity.service"

  - type: dt.entity.host
    relationship: "runsProcessGroup dt.entity.process_group"
```

Testing Segments

DQL: Segment Simulation

Simulate a host-group-based segment filter on log data:

```
// Filter logs by host group — simulates a segment filter condition on log data.
// `dt.host_group.id` is a Primary Grail Field that propagates across all signal types,
// so it is the canonical attribute to anchor a host-group-based segment on.
// The legacy `host.group` field is not populated on modern OneAgents and returns zero rows.
fetch logs, from:-1h
| filter isNotNull(dt.host_group.id)
| summarize count = count(), by:{dt.host_group.id}
| sort count desc
| limit 10
```

Segments and Access Control

Important: Segments are for **data filtering only**, not access control.

Function	Segments	IAM Policies
Filter data in queries	Yes	No
Restrict user access	No	Yes
Enforce security boundaries	No	Yes
Dynamic data views	Yes	No

Segments are governed by existing access controls - users only see data they're authorized to access.

Segment Design Checklist

- [ ] Defined clear segment purpose
- [ ] Identified all data types to include
- [ ] Verified metadata consistency (tags, host groups)
- [ ] Planned rule structure (max 20 includes)
- [ ] Considered entity relationships
- [ ] Tested filter conditions with DQL
- [ ] Documented segment usage

- [ ] Planned variable usage for flexibility

Bucket-Based Segment Filtering

Segments can filter by `dt.system.bucket` — making them useful for restricting dashboards and notebooks to specific storage buckets without modifying the query itself.

Static Bucket Segment

A segment that always targets a single bucket:

Field	Value
Data type	Logs
Filter condition	`dt.system.bucket = "default_logs"`

Dynamic Bucket Segment (with Variable)

Use a variable to make the bucket selection interactive at runtime:

```
Variable DQL:
fetch dt.system.buckets
| filter dt.system.table == "logs"
| fields bucket = name
| sort bucket asc
```

Then reference `$bucket` in the filter condition: `dt.system.bucket = $bucket`

This lets users pick a bucket from a dropdown in Dashboards or Notebooks without changing the underlying DQL query — useful for security reviews (pick the `audit_logs` bucket) or cost analysis (pick a specific team bucket).

> **Performance benefit:** targeting a bucket via segment reduces the data scanned compared to a full table scan, directly reducing query licensing costs.

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-09**: Enterprise Patterns
- **ORGNZ-10**: Advanced Segment Definitions

References

- Grail Segments
- Filter field (DT docs)
- Smartscape topology and entities (DT docs)
- Segment Limits
- Configure custom filter segments — get started (DT docs)

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.